

# Build Your Own Lock

(from Scratch)

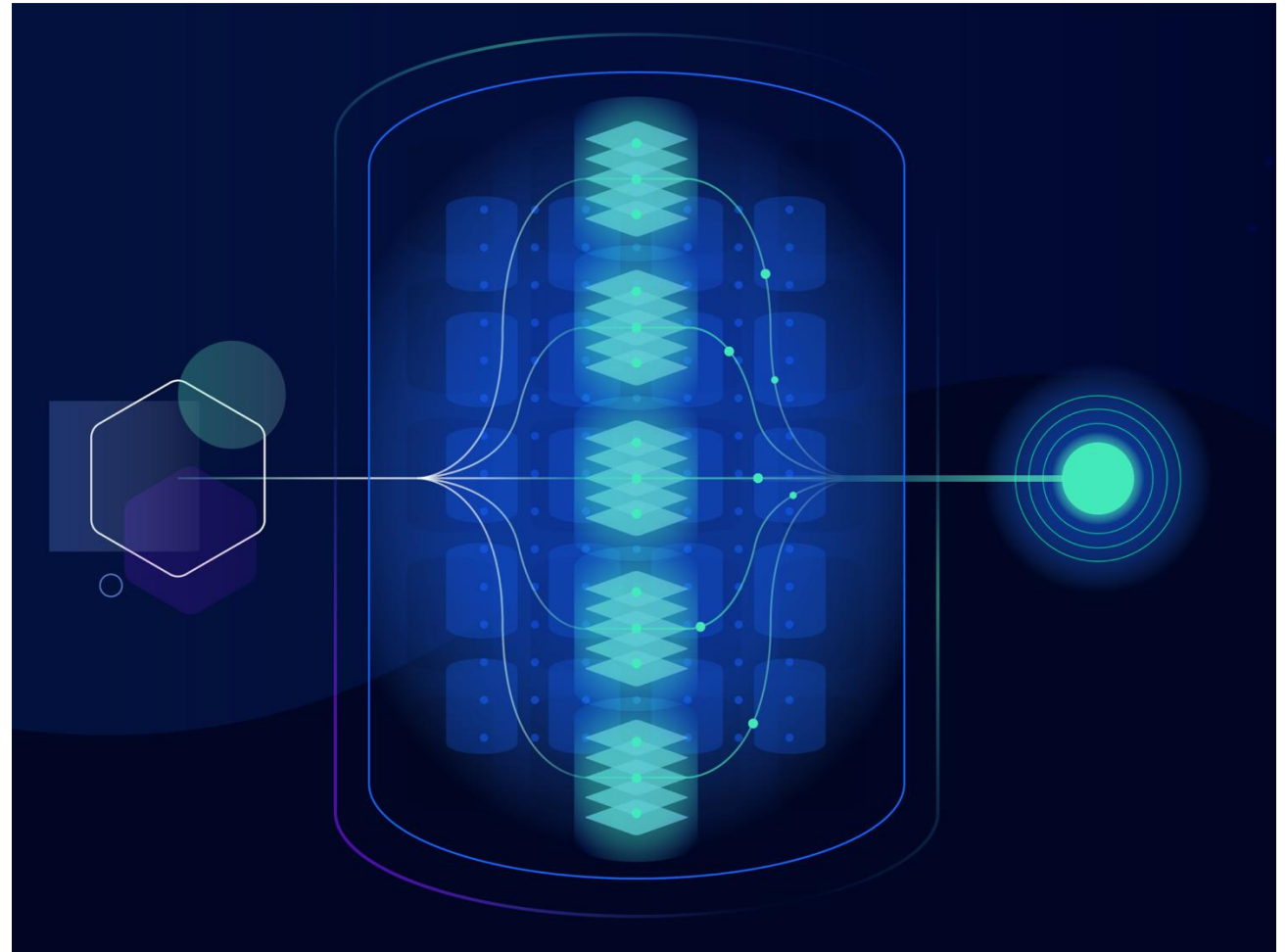


PRESENTER

Matthias Langer  
Dynatrace

# About me

- Senior Software Developer at Dynatrace
- Working **Grail™**



## Outline

---

### Introduction

- Lock what?

### Live Coding

- From 0 to a simplified version of `java.util.concurrent.locks.ReentrantLock`

### Wrapping up & Questions

- References & further information
- Q & A

# Introduction

# Lock what?

- Only one thread can hold the lock
- Locks protect resources from concurrent access
- They make sure that updates are visible across thread boundaries

```
interface Lock { 1 usage new *  
    void lock(); new *  
    void unlock(); new *  
}
```

```
final Lock lock; 3 usages  
int protectedResource; 1 usage
```

```
int useProtectedResource() { no usages new *  
    lock.lock();  
    try {  
        return protectedResource++;  
    } finally {  
        lock.unlock();  
    }  
}
```

## Standard choices in Java



### Synchronized keyword

- Can be used with arbitrary objects & has a nice syntax

### `java.util.concurrent.locks.ReentrantLock`

- More powerful & typically faster
- Usage less ergonomic

# Should I really build my own lock?

Don't!

But:

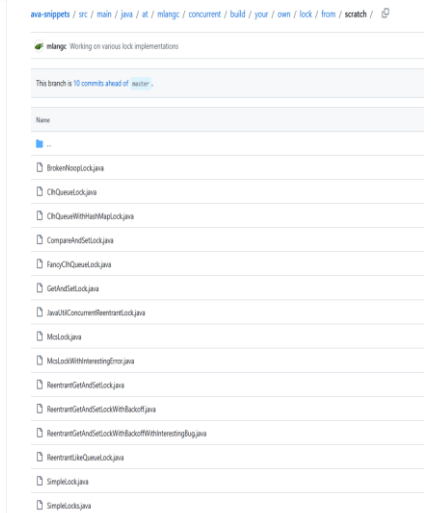
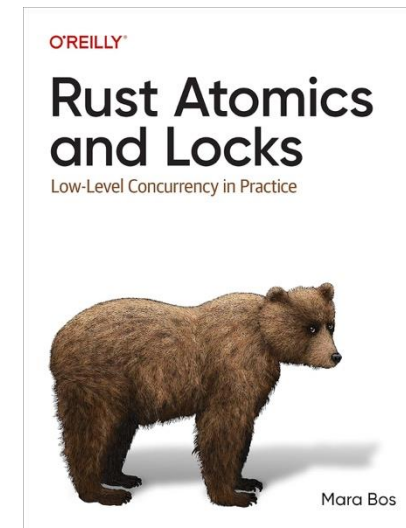
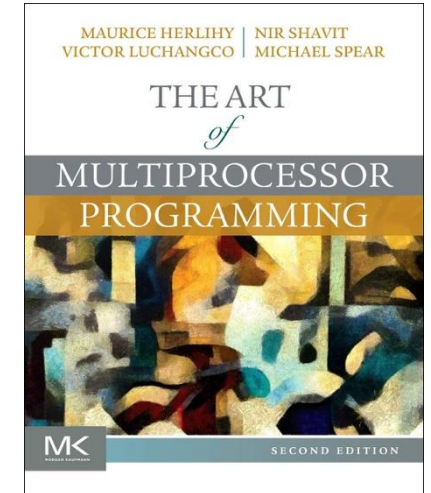
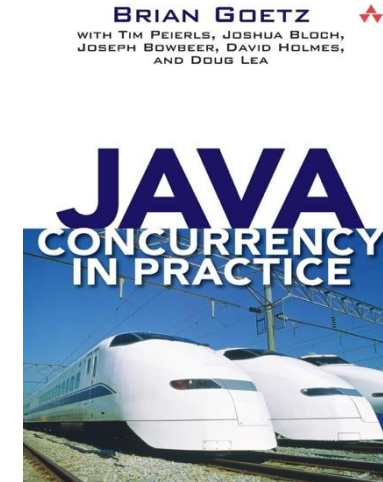
- Understanding locks helps understanding concurrency primitives in general
- They are widely known & have a simple interface
- Short algorithms appropriate for presentations exist
- Implementations can be tested & benchmarked without too much effort

Live Coding



# References

- The Art of Multiprocessor Programming 2nd Edition
- Java Concurrency in Practice
- Rust Atomics and Locks: Low-Level Concurrency in Practice
- [Close Encounters of The Java Memory Model Kind](#)
- [My Personal Blog](#)
- Personal [Java-Snippets Git Repo](#)



Q & A